



개와 고양이의 얼굴 영역 감지

☼ 상태	완료
📅 마감일	@11/27/2025
≡ 설명	스프린트 미션 #7

스프린트 미션 #7

개와 고양이의 얼굴 영역 감지

- 이번 미션에서는 SSD 모델을 활용하여 개와 고양이의 얼굴(Face) 영역을 감지하는 Object Detection 작업을 수행해 봅시다.
- 평가 지표(mAP, IOU 등)를 활용해 모델 성능을 분석하고 비교해 보세요.
- 사용 데이터셋 : [데이터 링크](#)(The Oxford-IIIT Pet Dataset)
 - annotations(xml): 각 이미지 파일에 대한 annotation
 - images: 이미지 파일(37종의 개와 고양이)

데이터 확인 및 전처리

1. 데이터 확인

1-1) 데이터 로드 / 경로 설정

```
# 데이터 경로 설정
path = "archive"

# 파일 경로 설정
trainval_file_path = "archive/annotations/annotations/trainval.txt"
test_file_path = "archive/annotations/annotations/test.txt"
```

```
# 이미지, Annotation 경로 설정
image_dir = "archive/images/images/"
xml_dir = "archive/annotations/annotations/xmls/"

# Train/Validation 파일 읽기
df_trainval = pd.read_csv(trainval_file_path, sep=r"\s+", header=None)
df_trainval.columns = ["Image", "ClassID", "Species", "BreedID"]

# Test 파일 읽기
df_test = pd.read_csv(test_file_path, sep=r"\s+", header=None)
df_test.columns = ["Image", "ClassID", "Species", "BreedID"]
```

1-2) 데이터 확인

```
# 데이터 크기 확인
print(f"Train/Validation 데이터 수: {len(df_trainval)}")
print(f"Test 데이터 수: {len(df_test)}")

# Annotation 개수 확인
xml_files = [file for file in os.listdir(xml_dir) if file.endswith(".xml")]
print(f"XML 파일 개수: {len(xml_files)}")
```

- 훈련 / 검증 데이터와 테스트 데이터의 개수 파악
xml 파일 개수와 차이가 있음.

```
print(df_trainval.shape)
df_trainval.head()

# Train과 Validation에 사용될 이미지 파일 이름 리스트 생성
trainval_list = df_trainval['Image'].tolist()

# Test에 사용될 이미지 파일 이름 리스트 생성
test_list = df_test['Image'].tolist()
```

	Image	ClassID	Species	BreedID
0	Abyssinian_100	1	1	1
1	Abyssinian_101	1	1	1
2	Abyssinian_102	1	1	1
3	Abyssinian_103	1	1	1
4	Abyssinian_104	1	1	1

```
# xml 파일 리스트
xml_list = [os.path.splitext(file)[0] for file in os.listdir(xml_dir) if file.endswith(".xml")]

# Train 이미지에 대해 XML 파일이 없는 경우 확인
missing_xml = [image for image in trainval_list if image not in xml_list]

# Train 이미지에 대해 XML 파일이 있는 경우 확인
trainval_list = [image for image in trainval_list if image in xml_list]

# 결과 출력
print(f"XML 파일이 없는 Train 이미지 수: {len(missing_xml)}")
print(missing_xml)
```

1-3) xml 데이터 확인

```
# 예제 XML 파일 경로
example_xml_file = os.path.join(xml_dir, xml_files[0])

# XML 파일 읽기 및 파싱
tree = ET.parse(example_xml_file)
root = tree.getroot()

# 재귀적으로 모든 태그와 데이터 출력 함수
def print_all_elements(element, indent=""):
    print(f"{indent}{element.tag}: {element.text}")
    for child in element:
        print_all_elements(child, indent + " ")
```

```
# XML 구조 탐색
print_all_elements(root)
```

```
# XML 파일에서 Bounding Box와 클래스 정보 추출
for obj in root.findall("object"):
    class_name = obj.find("name").text # 클래스 이름
    bndbox = obj.find("bndbox")
    x_min = int(bndbox.find("xmin").text)
    y_min = int(bndbox.find("ymin").text)
    x_max = int(bndbox.find("xmax").text)
    y_max = int(bndbox.find("ymax").text)

    print(f"Class: {class_name}, Bounding Box: ({x_min}, {y_min}, {x_max}, {y_max})")
```

```
# 모든 XML 파일 처리
annotations = []

for xml_file in xml_files:
    xml_path = os.path.join(xml_dir, xml_file)
    tree = ET.parse(xml_path)
    root = tree.getroot()

    image_name = root.find("filename").text # 이미지 파일 이름

    for obj in root.findall("object"):
        class_name = obj.find("name").text
        bndbox = obj.find("bndbox")
        x_min = int(bndbox.find("xmin").text)
        y_min = int(bndbox.find("ymin").text)
        x_max = int(bndbox.find("xmax").text)
        y_max = int(bndbox.find("ymax").text)

        annotations.append({
            "image": image_name,
```

```

        "class": class_name,
        "bbox": [x_min, y_min, x_max, y_max]
    })

```

1-4) 객체 인식 시각화

```

# Train 데이터에서 예제 이미지 불러오기
train_example_image_name = df_trainval["Image"].iloc[36]
train_image_path = os.path.join(image_dir, f"{train_example_image_name}.jpg")

# 이미지 읽기
image = cv2.imread(train_image_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

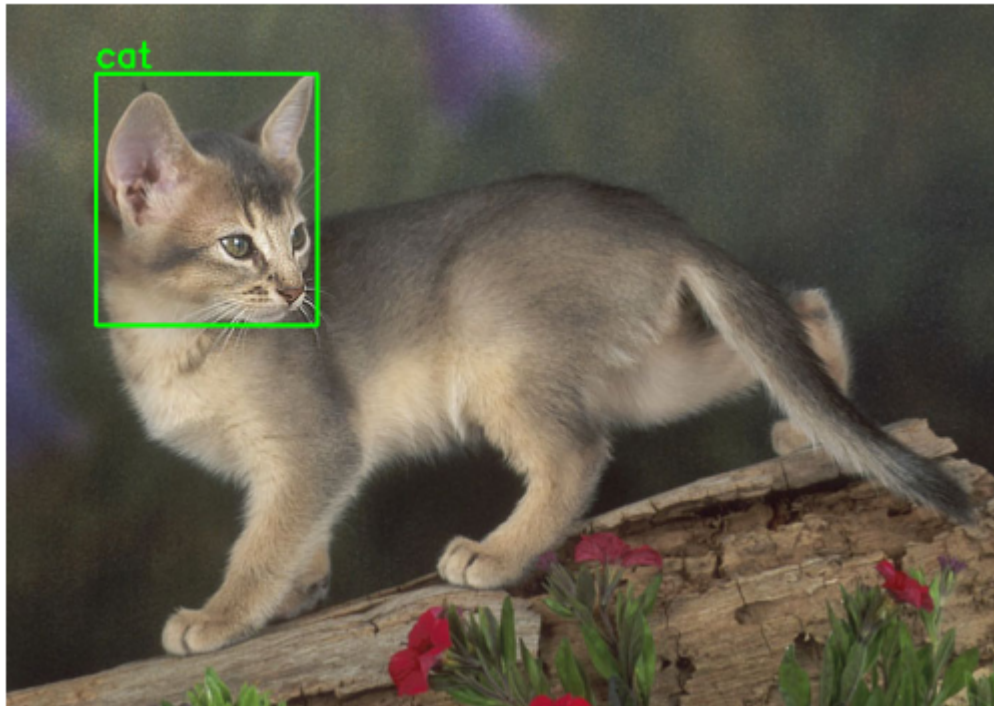
target_anno = [a for a in annotations if a["image"] == f"{train_example_image_name}.jpg"]

for ann in target_anno:
    x_min, y_min, x_max, y_max = ann["bbox"]

    cv2.rectangle(image, (x_min, y_min), (x_max, y_max), (0,255,0), 2)
    cv2.putText(image, ann["class"], (x_min, y_min - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

# 시각화
plt.imshow(image)
plt.axis("off")
plt.show()

```



2. 데이터 전처리

2-1) transform 정의

```
# transform 정의

transform = v2.Compose(
    [
        v2.ToImage(),
        v2.ToDtype(dtype=torch.float32, scale=True), # 정규화
    ]
)

# 클래스 정의
classes = ["background", "dog", "cat"]
```

2-2) 데이터셋 정의

```

from PIL import Image
from torch.utils.data import Dataset

# train 데이터셋

class VOCDataset(Dataset):
    def __init__(self, image_dir, annotation_dir, classes, image_list, transforms
=None):
        self.image_dir = image_dir
        self.annotation_dir = annotation_dir # xml_dir
        self.classes = classes # 정의된 classes
        self.transforms = transforms # 정의된 transform
        self.image_files = image_list # trainval_list를 train_list / val_list 로 split

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        # 이미지 및 XML 파일 경로 설정
        image_file = self.image_files[idx] + ".jpg"
        annotation_file = self.image_files[idx] + ".xml"
        image_path = os.path.join(self.image_dir, image_file)
        annotation_path = os.path.join(self.annotation_dir, annotation_file)

        # 이미지 로드
        image = Image.open(image_path).convert("RGB")

        # 어노테이션 로드
        boxes = []
        labels = []
        tree = ET.parse(annotation_path)
        root = tree.getroot()

        for obj in root.findall("object"):
            class_name = obj.find("name").text
            if class_name not in self.classes: # class_name이 정의된 classes에 없
으면 지나가라
                continue

```

```

        labels.append(self.classes.index(class_name))

        bndbox = obj.find("bndbox")
        x_min = int(bndbox.find("xmin").text)
        y_min = int(bndbox.find("ymin").text)
        x_max = int(bndbox.find("xmax").text)
        y_max = int(bndbox.find("ymax").text)
        boxes.append([x_min, y_min, x_max, y_max])

    # Tensor로 변환
    boxes = torch.tensor(boxes, dtype=torch.float32)
    labels = torch.tensor(labels, dtype=torch.int64)

    # Transform 적용
    if self.transforms:
        image, boxes, labels = self.transforms(image, boxes, labels)

    target = {"boxes": boxes, "labels": labels}
    return image, target

# test 데이터셋

class TestDataset(Dataset):
    def __init__(self, image_dir, image_list, transforms=None):
        self.image_dir = image_dir
        self.transforms = transforms
        self.image_files = image_list # 테스트 이미지 리스트 (확장자 없음)

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        # 이미지 파일 경로
        image_file = self.image_files[idx] + ".jpg"
        image_path = os.path.join(self.image_dir, image_file)

        # 이미지 로드
        image = Image.open(image_path).convert("RGB")

```



```

# Transform 적용
if self.transforms:
    image = self.transforms(image)

return image, self.image_files[idx] # 이미지와 파일 이름 반환

```

```

from sklearn.model_selection import train_test_split

# Train/Validation 분리 (trainval_list에서 80% Train, 20% Validation으로 나눔)
train_list, valid_list = train_test_split(trainval_list, test_size=0.2, random_state=42)

# 결과 확인
print(f"Train 이미지 수: {len(train_list)}")
print(f"Validation 이미지 수: {len(valid_list)}")
print(f"Test 이미지 수: {len(test_list)}")

```

2-3) 데이터로더 정의

```

from torch.utils.data import DataLoader

def ssd_collate_fn(batch):
    image, target = list(zip(*batch)) # zip(*x) → image끼리, target 끼리 모아줌
    return list(image), (target)

# Train Dataset
train_dataset = VOCDataset(
    image_dir=image_dir,
    annotation_dir=xml_dir,
    classes=classes,
    image_list=train_list, # Train 리스트 사용
    transforms=transform
)

# Validation Dataset
valid_dataset = VOCDataset(

```

```

    image_dir=image_dir,
    annotation_dir=xml_dir,
    classes=classes,
    image_list=valid_list, # Validation 리스트 사용
    transforms=transform
)

# Test Dataset 생성
test_dataset = TestDataset(
    image_dir=image_dir, # 테스트 이미지 디렉토리
    image_list=test_list, # 테스트 이미지 리스트 (확장자 없는 이름)
    transforms=transform # 필요하면 Transform 적용
)

# 데이터 로더 생성
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True, collate_fn=ssd_collate_fn)
val_loader = DataLoader(valid_dataset, batch_size=8, shuffle=False, collate_fn=ssd_collate_fn)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False, collate_fn=ssd_collate_fn)

# 데이터 크기 출력
print(f"Train 데이터셋 크기: {len(train_dataset)}")
print(f"Validation 데이터셋 크기: {len(valid_dataset)}")
print(f"Test 데이터셋 크기: {len(test_dataset)}")

```

객체 탐지 모델 SSD(Single Shot Multibox Detector)

1. SSD 모델 정의

```

# SSD 모델 불러오기
model = torchvision.models.detection.ssd300_vgg16(
                                weights=SSD300_VGG16_Weights.DE
                                FAULT).to(device)
model.train()

# 클래스 개수에 맞게 출력 레이어 수정
num_classes = len(classes) # background 포함
model.head.classification_head.num_classes = num_classes

# 옵티마이저 정의
optimizer = torch.optim.SGD(
                                model.parameters(), lr=0.001, momentum=0.9, weight_d
                                ecay=0.0005)

```

2. 계산 및 검증 함수 정의

2-1) IoU 계산 함수

```

# IOU 계산 함수
def calculate_iou(box, boxes):

    x_min = np.maximum(box[0], boxes[:, 0])
    y_min = np.maximum(box[1], boxes[:, 1])
    x_max = np.minimum(box[2], boxes[:, 2])
    y_max = np.minimum(box[3], boxes[:, 3])

    intersection = np.maximum(0, x_max - x_min) * np.maximum(0, y_max -
y_min)
    box_area = (box[2] - box[0]) * (box[3] - box[1])
    boxes_area = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
    union = box_area + boxes_area - intersection

    iou = intersection / union
    return iou

```

2-2) AP 계산 함수

```
# AP 계산 함수
def calculate_ap(predictions, ground_truths, class_idx, iou_threshold=0.5):

    true_positives = []
    false_positives = []
    all_ground_truths = 0

    # 모든 예측과 정답을 순회
    for pred, gt in zip(predictions, ground_truths):
        pred_boxes = np.array(pred["boxes"])
        pred_labels = np.array(pred["labels"])
        gt_boxes = np.array(gt["boxes"])
        gt_labels = np.array(gt["labels"])

        # 현재 클래스에 해당하는 박스만 필터링
        pred_boxes = pred_boxes[pred_labels == class_idx]
        gt_boxes = gt_boxes[gt_labels == class_idx]

        all_ground_truths += len(gt_boxes)

        # IoU 계산
        detected = []
        for pred_box in pred_boxes:
            ious = []
            for gt_box in gt_boxes:
                iou = calculate_iou(pred_box, gt_box)
                ious.append(iou)

            if len(ious) > 0:
                max_iou_idx = np.argmax(ious)
                if ious[max_iou_idx] >= iou_threshold and max_iou_idx not in detected:
                    true_positives.append(1)
                    false_positives.append(0)
                    detected.append(max_iou_idx)
            else:
```

```

        true_positives.append(0)
        false_positives.append(1)
    else:
        false_positives.append(1)

# Precision-Recall Curve 계산
tp_cumsum = np.cumsum(true_positives)
fp_cumsum = np.cumsum(false_positives)
precisions = tp_cumsum / (tp_cumsum + fp_cumsum + 1e-6)
recalls = tp_cumsum / (all_ground_truths + 1e-6)

# AP 계산
ap = 0.0
for i in range(1, len(precisions)):
    ap += (recalls[i] - recalls[i - 1]) * precisions[i]

return ap

```

2-3) 검증 함수

```

# 검증 함수
def evaluate_model(predictions, ground_truths, classes):
    class_aps = []

    for class_idx, class_name in enumerate(classes[1:], start=1):
        true_positives = []
        scores = []
        num_ground_truths = 0

        for pred, gt in zip(predictions, ground_truths):
            # Filter for the current class
            pred_boxes = pred["boxes"][pred["labels"] == class_idx].cpu().numpy() if len(pred["boxes"]) > 0 else []
            pred_scores = pred["scores"][pred["labels"] == class_idx].cpu().numpy() if len(pred["scores"]) > 0 else []
            gt_boxes = gt["boxes"][gt["labels"] == class_idx].cpu().numpy() if len(gt["boxes"]) > 0 else []

```

```

num_ground_truths += len(gt_boxes)

if len(pred_boxes) == 0 or len(gt_boxes) == 0:
    continue # Skip if no predictions or ground truths for this class

matched = np.zeros(len(gt_boxes), dtype=bool)
for box, score in zip(pred_boxes, pred_scores):
    ious = calculate_iou(box, gt_boxes)
    max_iou_idx = np.argmax(ious) if len(ious) > 0 else -1
    max_iou = ious[max_iou_idx] if max_iou_idx >= 0 else 0

    if max_iou >= 0.5 and not matched[max_iou_idx]:
        true_positives.append(1)
        matched[max_iou_idx] = True
    else:
        true_positives.append(0)

    scores.append(score)

if len(scores) == 0:
    class_aps.append(0)
    continue

sorted_indices = np.argsort(-np.array(scores))
true_positives = np.array(true_positives)[sorted_indices]
scores = np.array(scores)[sorted_indices]

cum_true_positives = np.cumsum(true_positives)
precision = cum_true_positives / (np.arange(len(true_positives)) + 1)
recall = cum_true_positives / num_ground_truths

ap = average_precision_score(true_positives, scores) if len(scores) >
0 else 0
class_aps.append(ap)

mAP = np.mean(class_aps)
return mAP

```

3. 모델 학습 및 검증

3-1) 학습 및 검증

```
# Training + Validation Loop
num_epochs = 5

for epoch in range(num_epochs):
    print(f"Epoch {epoch + 1}/{num_epochs} 시작")

    # Training Phase
    model.train()
    total_train_loss = 0

    for images, targets in tqdm(train_loader, desc="Training"):
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        # Forward pass
        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())
        total_train_loss += losses.item()

        # Backward pass and optimization
        optimizer.zero_grad()
        losses.backward()
        optimizer.step()

    avg_train_loss = total_train_loss / len(train_loader)
    print(f"Epoch {epoch + 1}/{num_epochs}, Train Loss: {avg_train_loss:.4f}")

    # Validation Phase
    model.eval()
    all_predictions = []
    all_ground_truths = []
```

```

with torch.no_grad():
    for images, targets in tqdm(val_loader, desc="Validation"):
        images = [img.to(device) for img in images]
        predictions = model(images)

        # 저장: 추론 결과와 Ground Truth
        all_predictions.extend(predictions)
        all_ground_truths.extend(targets)

# 성능 평가 (예: mAP 계산)
mAP = evaluate_model(all_predictions, all_ground_truths, classes)
print(f"Epoch {epoch + 1}/{num_epochs}, Validation mAP: {mAP:.4f}\n")

```

3-2) 모델 시각화 함수 정의

```

def visualize_prediction(image, prediction, classes):
    """
    image (torch.Tensor): 추론에 사용된 이미지 (C, H, W 형식).
    prediction (dict): 모델의 예측 결과 (boxes, labels, scores 포함).
    classes (list): 클래스 이름 리스트.
    """
    # Tensor 이미지를 (H, W, C) 형식으로 변환
    image = image.permute(1, 2, 0).numpy()

    # Matplotlib을 사용한 이미지 시각화
    fig, ax = plt.subplots(1, figsize=(8, 8))
    ax.imshow(image)

    # Bounding Box와 클래스 이름 시각화
    for box, label, score in zip(prediction["boxes"], prediction["labels"], prediction["scores"]):
        if score > 0.5: # Confidence Score 임계값
            x_min, y_min, x_max, y_max = box.tolist()
            width, height = x_max - x_min, y_max - y_min

            # Bounding Box 추가
            rect = patches.Rectangle(

```



```

        (x_min, y_min), width, height, linewidth=2, edgecolor="red", facec
olor="none"
    )
    ax.add_patch(rect)

    # 클래스 이름과 Confidence Score 추가
    ax.text(
        x_min,
        y_min - 10,
        f"{classes[label]}: {score:.2f}",
        color="red",
        fontsize=10,
        bbox=dict(facecolor="white", alpha=0.7),
    )

plt.axis("off")
plt.show()

```

3-3) 모델 검증 및 시각화

```

model.eval()
max_visualizations = 5 # 최대 시각화 개수 설정
count = 0

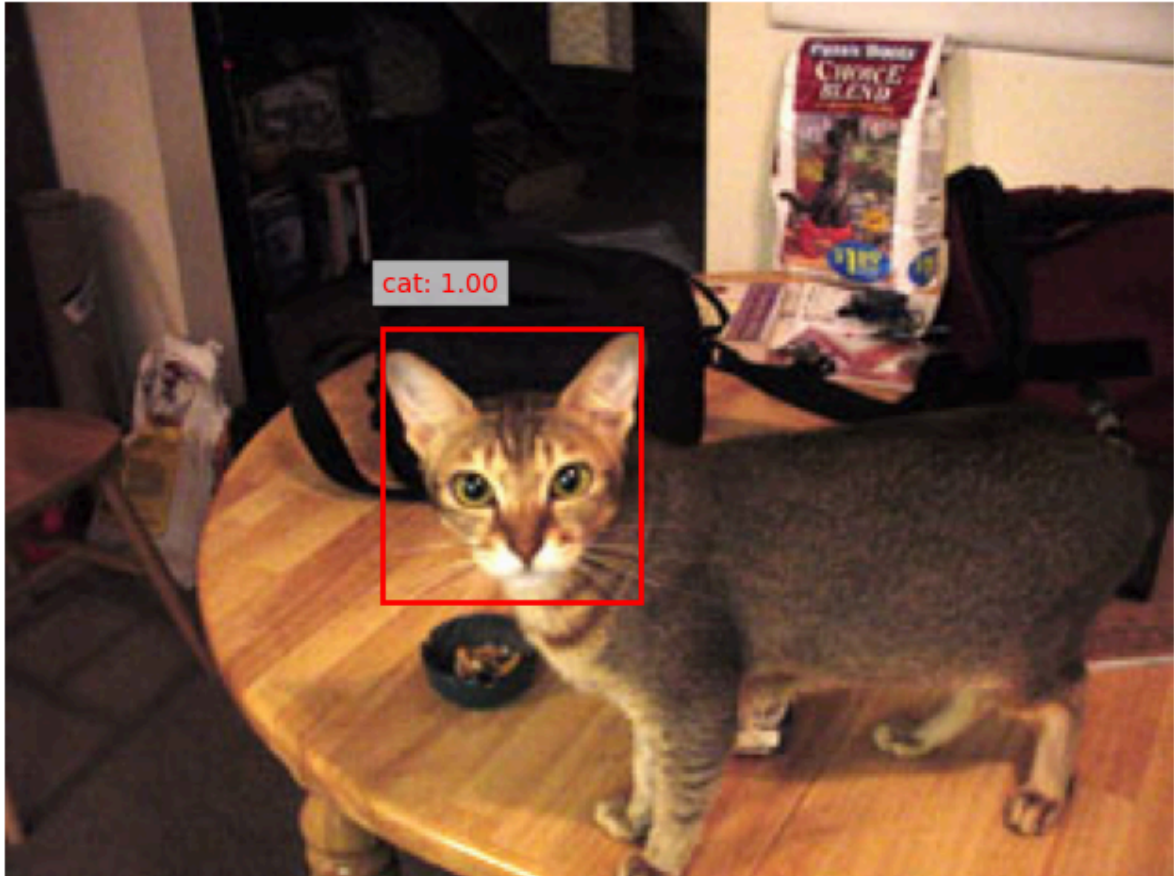
with torch.no_grad():
    for images, image_files in tqdm(test_loader, desc="Test Inference"):
        images = [img.to(device) for img in images]
        predictions = model(images)

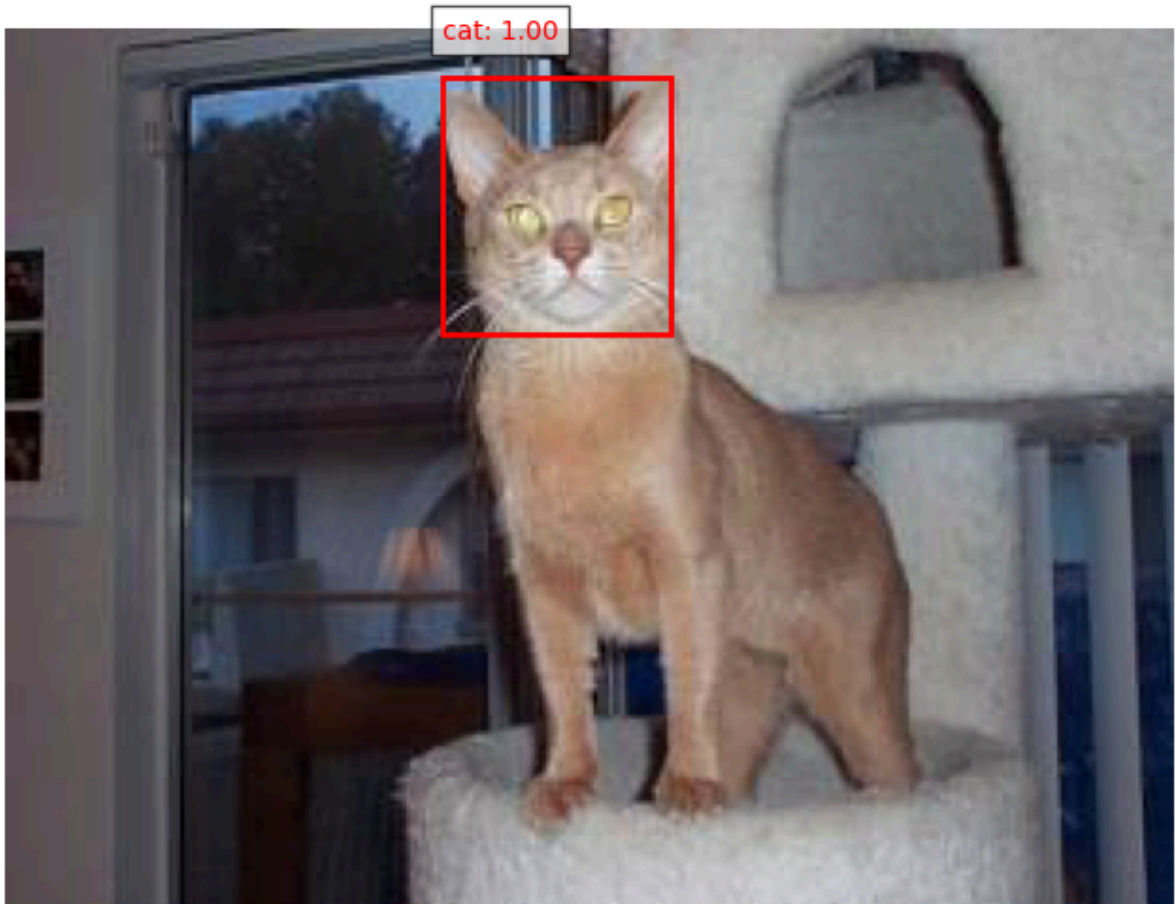
    for img, pred, file_name in zip(images, predictions, image_files):
        if count >= max_visualizations:
            break # 최대 개수 초과 시 중단

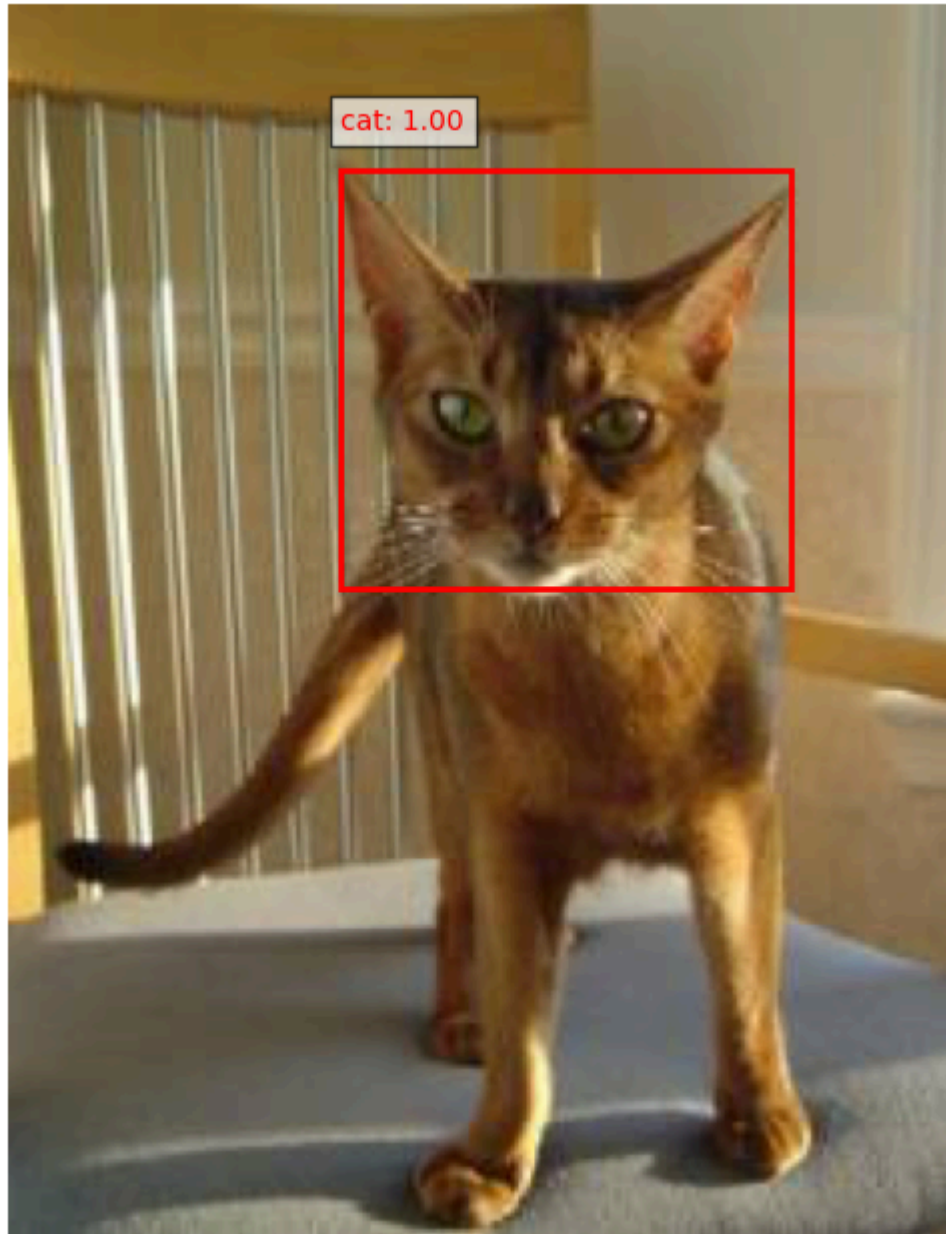
        print(f"Processing: {file_name}")
        visualize_prediction(img.cpu(), pred, classes)
        count += 1

```

```
if count >= max_visualizations:  
    break # 루프 종료
```







cat: 0.96



cat: 1.00

